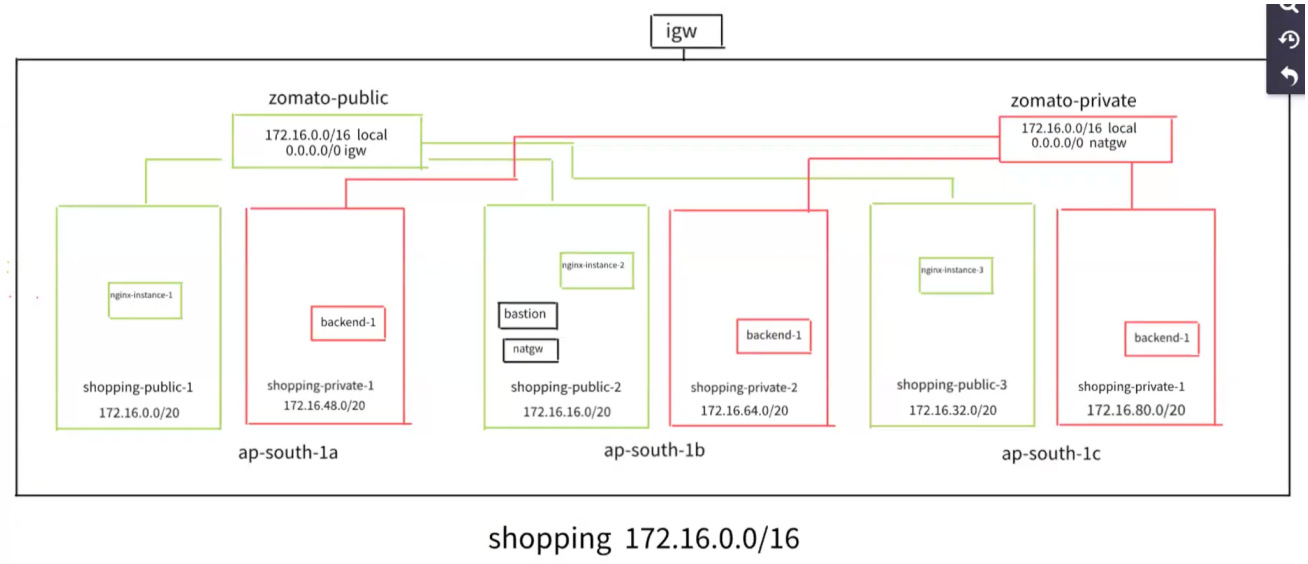
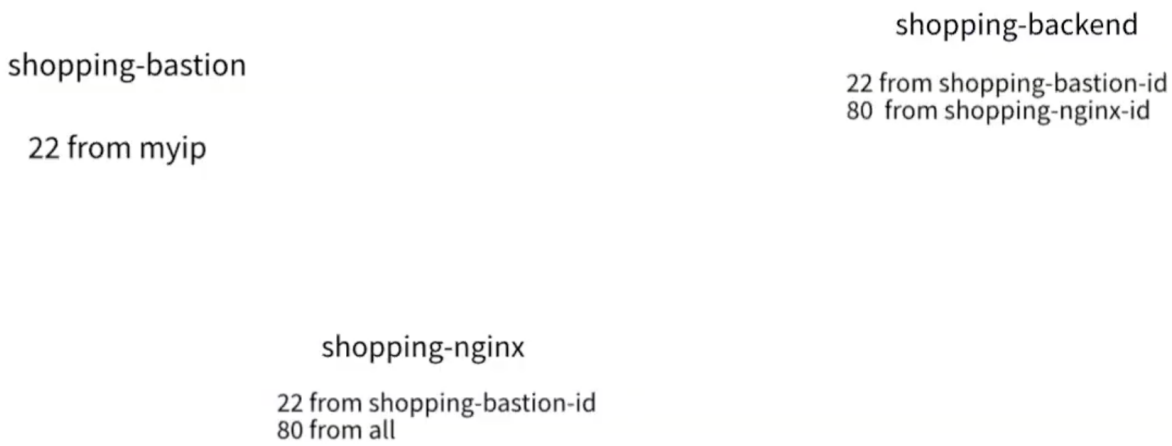
# NGINX-Based Load Balancer Project on AWS

This project demonstrates a custom load balancing architecture on AWS using NGINX instead of the AWS-managed Elastic Load Balancer (ELB). The infrastructure was designed manually by creating a custom VPC, public and private subnets, routing tables, NAT Gateway, and secure communication between frontend and backend servers.

## Architecture Diagram




## Security Group Configuration




## VPC Configuration

**VPC CIDR:** 172.16.0.0/16
**Region:** ap-south-1
**Availability Zones:** ap-south-1a, ap-south-1b, ap-south-1c

## Public Subnets

shopping-public-1 : 172.16.0.0/20
shopping-public-2 : 172.16.16.0/20
shopping-public-3 : 172.16.32.0/20

## Private Subnets

shopping-private-1 : 172.16.48.0/20
shopping-private-2 : 172.16.64.0/20
shopping-private-3 : 172.16.80.0/20

## NGINX Load Balancer

NGINX servers were deployed inside public subnets and configured as reverse proxy load balancers. Incoming traffic is distributed across multiple backend Apache servers using round-robin balancing.

## SSL/TLS Configuration

HTTPS was configured using Let's Encrypt SSL certificates for the domain:
**shopping.vpkdevops.online**

## Bastion Host

A bastion host was deployed inside the public subnet to securely access private backend servers through SSH.

## NGINX Load Balancer Configuration

```
upstream my_backends {

  server 172.16.55.62:8080;
  server 172.16.67.251:8080;
  server 172.16.84.35:8080;
}

server {

  listen 80;
  server_name _default_;

  location / {
    proxy_pass http://my_backends;

    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-Port $server_port;
  }
}
```

## Backend Apache HTTPD Setup

```
sudo yum install httpd php-fpm git -y
sudo systemctl restart httpd.service php-fpm.service
sudo systemctl enable httpd.service php-fpm.service

sudo git clone https://github.com/Fujikomalan/aws-elb-site
sudo cp -R aws-elb-site/* /var/www/html/

sudo chown -R apache:apache /var/www/html/*
sudo vim /var/www/html/index.php
```

```
sudo netstat -ntlp

sudo systemctl restart httpd.service

sudo vim /etc/httpd/conf/httpd.conf

LogFormat "%h %{X-Forwarded-For}i %{remote}p %{X-Forwarded-Proto}i %{X-Forwarded-Port}i %l %u %t \"%r\" %>s %b

sudo systemctl restart httpd.service
```

## Security Group Rules

| Security Group | Ports | Source |
|---|---|---|
| shopping-bastion | 22 | My Public IP |
| shopping-nginx | 22 | shopping-bastion-id |
| shopping-nginx | 80/443 | 0.0.0.0/0 |
| shopping-backend | 22 | shopping-bastion-id |
| shopping-backend | 80/8080 | shopping-nginx-id |

## Request Flow

```
Client Request
      ↓
shopping.vpkdevops.online
      ↓
NGINX Load Balancer (Public Subnet)
      ↓
Backend Apache Servers (Private Subnets)
      ↓
Application Response
      ↓
Client
```

## Conclusion

This project successfully implemented a secure and scalable custom load balancing architecture on AWS using NGINX instead of AWS-managed load balancers. The backend servers were isolated inside private subnets, secured using bastion hosts and security groups, and external communication was protected using HTTPS with Let’s Encrypt SSL certificates.